

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“MODUL 4 PRAKTIKUM”



disusun Oleh:
FILZI JELILA INDA ROBBANI
(2511533019)

Dosen Pengampu:
NURFIAH, S.ST, M.Kom

Asisten Praktikum:
DAWI SYAUQI

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

DAFTAR ISI

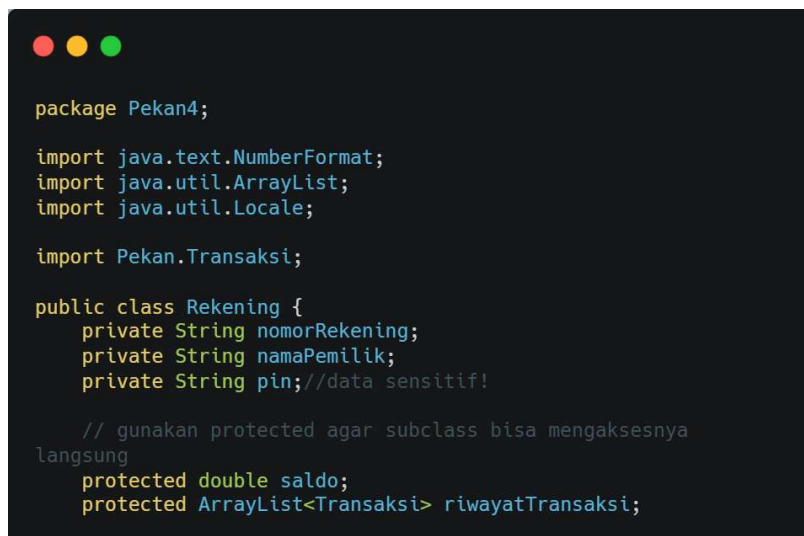
DAFTAR ISI.....	ii
BAB I PEMBAHASAN	12
1.1 Langkah Kerja Praktikum	12
1.1.1 Memastikan Akses Modifier di Rekening.java.....	12
1.1.2 Membuat Subclass RekeningTabungan	13
1.1.2 Membuat Subclass RekeningGiro	14
1.2 Latihan/Tugas Praktikum	15
1.2.1 Deskripsi Tugas	15
1.2.2 Pengerjaan Latihan/Tugas	15
1.2.3 Link Repository.....	18
BAB II PENUTUP	19
2.1 Kesimpulan	19
DAFTAR PUSTAKA.....	20

BAB I PEMBAHASAN

1.1 Langkah Kerja Praktikum

1.1.1 Memastikan Akses Modifier di Rekening.java

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru “Pekan4” dan class baru mengcopy semua class pada minggu lalu (rekening, transaksi, main).
- 2) Setelah itu buka class rekening ubah variabel saldo dan riwayatTransaksi menggunakan hak akses protected, sementara pin dan nomorRekening tetap private. Untuk konstruktor dan method lainnya tetap/tidak ada perubahan.



```
package Pekan4;

import java.text.NumberFormat;
import java.util.ArrayList;
import java.util.Locale;

import Pekan.Transaksi;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private String pin;//data sensitif!

    // gunakan protected agar subclass bisa mengaksesnya
    langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;
```

Beberapa atribut tsb diubah menjadi protected agar data tersebut tetap terlindungi dari akses langsung oleh kelas lain, tetapi masih bisa diakses oleh subclass/kelas turunan seperti RekeningVIP. Jadi, penggunaan protected ini bertujuan supaya kelas turunan dapat menggunakan atau mengolah saldo dan riwayatTransaksi secara langsung tanpa harus membuat akses tambahan, sekaligus tetap menjaga konsep enkapsulasi karena tidak semua kelas bisa mengaksesnya secara bebas.

1.1.2 Membuat Subclass RekeningTabungan

- 1) Buat file/class baru bernama RekeningTabungan.java. Produk tabungan memiliki fitur tambahan yaitu Suku Bunga Bulanan.

```
package Pekan4;
import Pekan.Transaksi;;

public class RekeningTabungan extends Rekening {
    //atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    //Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double
    saldoAwal,String pinAwal, double sukuBunga) {
        //super() memanggil constructor kelas induk (Rekening). WAJIB
        berada dibaris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga=sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        //menghitung bunga
        //mengapa bisa mengakses saldo secara langsung dari class
        RekeningTabungan?
        double nominalBunga = saldo * (sukuBunga/100);
        saldo += nominalBunga;

        //mencatat riwayat Transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga",
        nominalBunga));

        System.out.println("Bunga "+sukuBunga + "% berhasil ditambahkan :
        Rp"+ nominalBunga);
    }
}
```

Class RekeningTabungan.java ini merupakan subclass dari class Rekening, terlihat dengan pemakaian perintah extends. Sehingga class ini mewarisi atribut dan method dari kelas induknya/superclass. Atribut tambahan sukuBunga dibuat private karena hanya digunakan khusus untuk rekeningTabungan saja.

Pada constructor, super(...) digunakan untuk memanggil constructor kelas Rekening terlebih dahulu, kemudian sukuBunga diisi melalui this.sukuBunga.

Method tambahBungaAkhirBulan() digunakan untuk menghitung bunga berdasarkan saldo dan sukuBunga, lalu hasil bunganya ditambahkan ke saldo. saldo dan riwayatTransaksi seperti yang sudah dijelaskan pada langkah 1 mereka bisa diakses langsung karena sebelumnya pada kelas Rekening diubah menjadi protected,

sehingga dapat digunakan oleh subclass seperti RekeningTabungan ini. Setelah bunga ditambahkan, transaksi bunga juga dicatat ke dalam riwayatTransaksi agar masuk ke riwayat rekening.

1.1.2 Membuat Subclass RekeningGiro

- 1) Buat file baru bernama RekeningGiro.java. Produk Giro digunakan pebisnis dan memiliki fitur tambahan yaitu Batas Overdraft (Limit Pinjaman).

```
package Pekan4;

public class RekeningGiro extends Rekening{
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
        //memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }
    //getter khusus giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Selanjutnya subclass RekeningGiro, sama seperti RekeningTabungan yaitu turunan dari class Rekening. Atribut batasOverdraft dibuat private karena merupakan data khusus yang hanya digunakan oleh rekening giro sama seperti atribut sukuBunga pada RekeningTabungan. Pada constructor, super(...) digunakan untuk memanggil constructor Rekening agar data dasar seperti nomor rekening, nama, saldo awal, dan PIN dapat diinisialisasi terlebih dahulu, kemudian this.batasOverdraft digunakan untuk menyimpan batas overdraft rekening giro. Method getBatasOverdraft() berfungsi sebagai getter untuk mengambil nilai batas overdraft karena atributnya bersifat private, sehingga akses terhadap data tersebut tetap dikontrol melalui method.

1.2 Latihan/Tugas Praktikum

1.2.1 Deskripsi Tugas

Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di Main.java agar mendukung produk-produk baru ini secara interaktif.

Tugas 1: Pembuatan Multi-Produk (40%)

Modifikasi case 1: Buka Rekening Baru di Main.java.

- Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | Giro Bisnis"
- Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
- Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
- Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Tugas 2: Simulasi Waktu Akhir Bulan : (60%)

- Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
- Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
- Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

1.2.2 Pengerjaan Latihan/Tugas

1) Tugas 1 :

Kode dikelas Main.java dimodifikasi agar sistem dapat membuat beberapa pilihan produk rekening, pilihan produk yang saya buat terdiri dari 3 pilihan, dimana diluar soal tugas

ada challenge yang saya kerjakan yaitu penambahan RekeningVIP.

Nah setelah nomor rekening, nama, saldo awal, dan PIN diinput serta PIN dinyatakan valid, user diberikan pilihan produk yaitu Tabungan Umum, Giro Bisnis, atau VIP.

```
//TUGAS 1
System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3.
VIP (+Bonus)");
System.out.print("Pilihan: ");
int pilihProduk = input.nextInt();
input.nextLine();

Rekening rekeningBaru = null;

if (pilihProduk == 1) {
    System.out.print("Masukkan suku bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();
    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
} else if (pilihProduk == 2) {
    System.out.print("Masukkan batas overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();
    rekeningBaru = new RekeningGiro(no, nama, saldo, pin,
    batasOverdraft);
} else if (pilihProduk == 3) {
    // Bonus Rp100.000 otomatis masuk dari konstruktor RekeningVIP
    rekeningBaru = new RekeningVIP(no, nama, saldo, pin);
} else {
    System.out.println("Pilihan produk tidak valid. Pembukaan rekening
    dibatalkan.");
}

if (rekeningBaru != null) {
    akunAktif = rekeningBaru;
    bukaRekening.add(rekeningBaru);
    System.out.println("Rekening berhasil dibuat!");
}

break;
```

Jika memilih Tabungan, user diminta memasukkan sukuBunga lalu dibuat objek RekeningTabungan, sedangkan jika memilih Giro, user diminta memasukkan batasOverdraft lalu dibuat objek RekeningGiro. Objek tersebut kemudian disimpan ke variabel Rekening rekeningBaru, sehingga terjadi upcasting, yaitu objek subclass seperti RekeningTabungan dan RekeningGiro dapat disimpan menggunakan tipe superclass Rekening. Setelah itu objek dimasukkan ke bukaRekening menggunakan add() dan dijadikan sebagai akunAktif. Jika memilih rekeningVIP, program akan membuat objek rekeningvip. Pada bagian ini tidak ada input tambahan seperti sukuBunga/ batasOverdraft karena fitur khusus VIP sudah

ditentukan di dalam class RekeningVIP, termasuk saldo Rp.100.000 yang otomatis diberikan melalui constructor. Setelah objek berhasil dibuat, objek menjadi akunAktif dan disimpan ke dalam daftar rekening sama seperti pilihan 1 atau 2.

2) Tugas 2 :

Untuk tugas ini sedikit berbeda jawaban saya dengan permintaan tugas, dimana saya membuat menu baru nya bukan di pilihan ke7 melainkan ke 9 seperti yang terlihat pada kode tersebut

```
while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Cetak Mutasi(Riwayat)");
    System.out.println("7. Riwayat 3 Transaksi terbaru");
    System.out.println("8. Ganti PIN");
    System.out.println("9. Simulasi Akhir Bulan (Khusus
    Tabungan)");
    System.out.println("0. Keluar");
    System.out.println("Pilih Menu : ");

    int pilihan = input.nextInt();
    input.nextLine();
}
```

```
case 9 :
//TUGAS 2
if (akunAktif == null) {
    System.out.println("Error : Mohon maaf, anda belum memiliki norek!");
} else if (akunAktif instanceof RekeningTabungan) {
    RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
    tabungan.tambahBungaAkhirBulan();
} else {
    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku
    untuk Rekening Tabungan.");
}
break;
```

Pada case 9 ini pertama program akan mengecek apakah `akunAktif==nul`, lalu menggunakan operator `instanceof` `RekeningTabungan` untuk memastikan akun yang sedang aktif merupakan objek `RekeningTabungan`. Karena pilihan menu 9 Simulasi Akhir Bulan ini hanya bisa digunakan oleh rekening tabungan. Jika benar, dilakukan downcasting dari tipe `Rekening`

menjadi RekeningTabungan melalui RekeningTabungan tabungan = (RekeningTabungan) akunAktif;, sehingga method khusus tambahBungaAkhirBulan() dapat dipanggil. Jika akun yang aktif bukan rekening tabungan, program menampilkan pesan gagal, bahwa fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

1.2.3 Link Repository

https://github.com/filzijelilair/PrakPBO_C_2511533019

BAB II

PENUTUP

2.1 Kesimpulan

Kesimpulan dari praktikum ini adalah bahwa penerapan inheritance, upcasting, dan downcasting dapat digunakan untuk membuat sistem perbankan dengan beberapa jenis produk rekening. Melalui RekeningTabungan, RekeningGiro, dan RekeningVIP, setiap rekening dapat memiliki fitur khusus masing-masing, tetapi tetap dapat dikelola melalui superclass Rekening. Praktikum ini juga menunjukkan penggunaan instanceof untuk mengecek tipe objek sebelum melakukan downcasting, sehingga fitur seperti penambahan bunga akhir bulan hanya dapat dijalankan pada rekening yang sesuai

DAFTAR PUSTAKA

- [1] Nurfiah, S.ST., M.Kom., *Modul Praktikum 2: Relasi Antar Objek (Class Diagram & Asosiasi)*.
- [2] Nurfiah, S.ST., M.Kom., *Pemrograman Berbasis Objek: Encapsulation*, Program Studi Informatika, Fakultas Teknologi Informasi, Universitas Andalas, Padang, 2026.